More practice - Heaps and top-k

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	2	1
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Heaps and top-k | 4 questions | tap a title to jump to it

E	1. Find k pairs with smallest sums	PracHub	2025-11-01
M	2. Implement frequency + distance top-K queries	PracHub	2026-02-12
M	3. Determine if a 14-tile hand is winning	PracHub	2026-03-09
H	4. Return top-k subset sums in descending order	PracHub	2026-01-07

STUDY PATH: question 110 of 167 (more practice)

Priority score 1.6 | Heaps and top-k | asked 353 days ago

EASY | IN APP LC 373 | PracHub | Technical Screen | 1 of 4

Find k pairs with smallest sums

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 119/228

ABOUT THIS QUESTION

Topic: More practice - Heaps and top-k

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #373 Find K Pairs with Smallest Sums

LeetCode match: LC 373 Find K Pairs with Smallest Sums

Posted on PracHub: 2025-11-01 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-k-pairs-with-smallest-sums

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **`k`** pairs **`(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17 The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

APPROACH

Min-heap seeded with (arr1[i]+arr2[0], i, 0) for the first min(k, len1) indices; pop, then push (i, j+1). O(k log k). Return all pairs if fewer than k exist.

HOW TO THINK ABOUT IT

Because both arrays are sorted, the pair (i, j + 1) is never smaller than (i, j), so each row i of the 'matrix of sums' is sorted. The k smallest pairs are the k smallest items of a collection of sorted rows, which is a heap problem: start with the first element of each row (or the first few) and each time you pop a pair, push its successor in the row.

Basis: LeetCode 373, Find K Pairs with Smallest Sums (two sorted arrays).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, b, k):
    pairs = [(x, y) for x in a for y in b]
    pairs.sort(key=lambda p: p[0] + p[1])
    return pairs[:k]
```

TIME	SPACE
O(m * n log(m * n))	O(m * n)

WHY IT IS SLOW

m * n pairs sorted: with 10^5 elements in each array that is 10^10 pairs. Because both arrays are sorted, a min-heap that starts from each x's first pair and expands only the next pair in that row is O(k log k).

OPTIMAL SOLUTION · Min-heap over the sorted rows

TIME	SPACE
O(k log min(k, m))	O(min(k, m))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

FOLLOW-UP 2 Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

FOLLOW-UP 3 Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. **Cost:** O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find k pairs with smallest sums
#
# You are given two **sorted** integer arrays arr1 and
# arr2 (each sorted in non-decreasing order), and an
# integer k. Consider all possible pairs formed by taking
# one element from arr1 and one element from arr2. For
# indices i and j, the pair is (arr1[i], arr2[j]) with sum
# arr1[i] + arr2[j]. Your task is to return the **k
# pairs** (arr1[i], arr2[j]) with the smallest sums. If
# there are fewer than k total possible pairs, return all
# possible pairs. You may return the pairs in any order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The arrays are not sorted.
# 2. Only the k-th smallest sum, with huge arrays.
# 3. Three arrays, or k pairs with the largest sums.
#
# INTERVIEW NOTES
# Because both arrays are sorted, the pair (i, j + 1) is
# never smaller than (i, j), so each row i of the 'matrix
# of sums' is sorted. The k smallest pairs are the k
# smallest items of a collection of sorted rows, which is
# a heap problem: start with the first element of each row
# (or the first few) and each time you pop a pair, push
# its successor in the row.
#
# BRUTE FORCE (say it first): Generate all pairs and sort
# Build all m * n pairs, sort by sum and take k. Correct
# and the oracle; with 10^5 elements in each array it
# would create 10^10 pairs. Time O(m * n log(m * n)).
# Space O(m * n).


import heapq
from bisect import bisect_right


# BRUTE FORCE (say it first): Generate all pairs and sort
# | time O(m * n log(m * n))
def k_pairs_all(a, b, k):
    return sorted(
        ((x, y) for x in a for y in b),
        key=lambda p: (p[0] + p[1], p),
    )[:k]
```

```
# HELPERS for follow-up 2
def count_at_most(a, b, limit):
    # pairs with sum <= limit: for every x binary search
    # in b
    return sum(bisect_right(b, limit - x) for x in a)


# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (a[i] + b[0], i, 0) for the first min(k, m)
# values of a. Pop the smallest, record the pair, and push
# the next pair in the same row, (a[i] + b[j + 1], i, j +
# 1). After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    # a and b sorted: a heap holds the best unused pair of
    # each row i; popping (i, j) offers (i, j + 1)
    if not a or not b or k <= 0:
        return []
    rows = range(min(k, len(a)))
    heap = [(a[i] + b[0], i, 0) for i in rows]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            nxt = a[i] + b[j + 1]
            heapq.heappush(heap, (nxt, i, j + 1))
    return out


# FOLLOW-UP 1: The arrays are not sorted.
# Answer: Sort both first (O(m log m + n log n)); the heap
# method needs sorted rows. For one huge array you may
# instead use a heap of size k with a selection of
# candidates.
def k_smallest_pairs_unsorted(a, b, k):
    # the heap method needs sorted rows: sort first
    return k_smallest_pairs(sorted(a), sorted(b), k)


# FOLLOW-UP 2: Only the k-th smallest sum, with huge
# arrays.
# Answer: Binary search the sum value and count how many
# pairs have sum at most it with two pointers (O(m + n)
```

```
# per check): O((m + n) log(range)) with O(1) space, no
# heap.
def kth_smallest_sum(a, b, k):
    # huge arrays: binary search the SUM, no heap needed
    lo, hi = a[0] + b[0], a[-1] + b[-1]
    while lo < hi:
        mid = (lo + hi) // 2
        if count_at_most(a, b, mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 3: Three arrays, or k pairs with the largest
# sums.
# Answer: For largest sums run the same algorithm on the
# reversed arrays (or negate the values). For three arrays
# use a heap of triples (i, j, l), pushing the three
# neighbours with a visited set.
def k_largest_pairs(a, b, k):
    # largest sums: negate the values and run the same
    # code
    neg_a = [-x for x in sorted(a, reverse=True)]
    neg_b = [-y for y in sorted(b, reverse=True)]
    pairs = k_smallest_pairs(neg_a, neg_b, k)
    return [(-x, -y) for x, y in pairs]


if __name__ == "__main__":
    a, b = [1, 7, 11], [2, 4, 6]
    three = k_smallest_pairs(a, b, 3)
    assert three == [(1, 2), (1, 4), (1, 6)]
    assert k_smallest_pairs(a, b, 4)[3] == (7, 2)
    assert k_smallest_pairs([1, 1, 2], [1, 2, 3], 2) == [
        (1, 1), (1, 1)]
    assert k_smallest_pairs([], [1], 3) == []
    # k bigger than the number of pairs: all of them
    assert len(k_smallest_pairs([1], [1, 2], 10)) == 2
    fixed = k_smallest_pairs_unsorted([7, 1], [4, 2], 2)
    assert fixed == [(1, 2), (1, 4)]
    assert k_largest_pairs(a, b, 2) == [(11, 6), (11, 4)]
    # the 4th smallest sum is 9 (7 + 2); check by sorting
    # all
    sums = sorted(x + y for x in a for y in b)
```

```
    for k in range(1, 10):
        assert kth_smallest_sum(a, b, k) == sums[k - 1]
```

STUDY PATH: question 111 of 167 (more practice)

Priority score 3.35 | Heaps and top-k | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 2 of 4

★ TOP 75: rank 28 in the Medium and Hard priority list

★ Implement frequency + distance top-K queries

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 20/228

ABOUT THIS QUESTION

Topic: More practice - Heaps and top-k

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #347 Top K Frequent Elements, LeetCode #973 K Closest Points to Origin

LeetCode match: LC 347 Top K Frequent Elements + LC 973 K Closest Points to Origin

Posted on PracHub: 2026-02-12 (updated 2026-04-28)

Original: https://prachub.com/coding-questions/implement-frequency-plus-distance-top-k-queries

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **x** ascending, then **y** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **k**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.

QUESTION

- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.
Constraints
1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1]
    appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]],
    3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each
    appear twice. Their squared distances are 4, 5,
    and 5, so [0,2] comes first. Between [1,2] and
    [2,1], x is smaller for [1,2].
```

QUESTION

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like `(-frequency, distance, x, y)`.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either `['set', key, value, timestamp]` or `['getClosest', key, timestamp]`. For every `getClosest` operation, return the stored value whose timestamp is closest to the query tim

APPROACH

Count frequencies in a hash map; use a size-k heap with key (frequency desc, distance asc) for Q1, and a max-heap of size k on distance for Q2.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two questions. Q1 (top-K points): confirm the full ordering (higher frequency, then smaller distance, then smaller x, then smaller y) and that K is at most the number of distinct points. Q2 (time-based store): timestamps for a key arrive in non-decreasing order, and a tie in distance returns the smaller timestamp.

BRUTE FORCE TO SAY OUT LOUD

Q1: sort all distinct points with the full key. Q2: linear scan of a key's timestamps. Both are correct and easy to state.

OPTIMAL IDEA

Q1: Counter over tuples, then heapq.nsmallest(k, distinct, key=(-freq, x*x + y*y, x, y)), O(n + d log k). Q2: per key keep parallel lists of timestamps and values; bisect_left finds the first timestamp at least the query; compare it with the previous one and return the closer value, the smaller timestamp on a tie. O(log m) per query.

DRY-RUN THESE CASES

Q1: duplicates, ties on frequency and distance, K equal to the number of distinct points. Q2: query before the first timestamp, after the last, exactly on a timestamp, the tie example (a, 12 returns y), an unknown key.

LIKELY FOLLOW-UPS

Q1: top-K in a stream. Q2: an out-of-order insert (then use a sorted container), deletes, and memory limits.

PITFALLS

Q1: sorting by distance only. Q2: off-by-one at the ends of the list.

HOW TO THINK ABOUT IT

Q1 is a ranking: say the full sort key out loud, count with a hash map, and then you only need the K best (heap), not a full sort. Q2 keeps each key's timestamps in a sorted list, so the nearest one is next to the insertion point: binary search, then compare the two neighbours.

Basis: Two tasks: top K points by (frequency desc, squared distance asc, x asc, y asc), a mix of LeetCode 347 and 973; and a time-based store whose getClosest returns the value at the nearest timestamp (ties to the smaller one), a variant of LeetCode 981.

BRUTE FORCE · Q1: sort all distinct points by the full key

BRUTE FORCE · PSEUDO-CODE

```
def top_k_points(points, k):
    counts = Counter(map(tuple, points))
    def rank(p):
        x, y = p
        return (-counts[p], x * x + y * y, x, y)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct points, O(d log d); a size-K heap with the same key is O(d log k).

BRUTE FORCE · Q2: scan every timestamp of the key

BRUTE FORCE · PSEUDO-CODE

```
def get_closest(key, t):
    best = None
    for ts, value in store[key]:
        gap = abs(ts - t)
        if best is None or (gap, ts) < best[:2]:
            best = (gap, ts, value)
    return best[2]
```

TIME	SPACE
O(m) per query	O(total entries)

WHY IT IS SLOW

Linear in the number of stored timestamps per query. Timestamps arrive in order, so binary search on the sorted list finds the neighbours in O(log m).

OPTIMAL SOLUTION · Q1: heap with the full key

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: parallel sorted lists and bisect

TIME	SPACE
O(1) per set, O(log m) per getClosest	**O(total entries)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log k) per query), or maintain a sorted structure updated on every add if queries dominate.

FOLLOW-UP 2 Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

FOLLOW-UP 3 Q2 returns the closest value at or before the timestamp (LeetCode 981).

Use bisect_right(timestamps, ts) - 1 and return that value or an empty string if the index is negative; there is no tie question.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement frequency + distance top-K
# queries
#
# You will implement solutions for two coding interview
# questions. ## Question 1: Return the top-K points by
# frequency (with distance tie-break) You are given an
# array points of 2D integer points, where each point is
# represented as [x, y]. The array may contain duplicates
# (the same [x, y] can appear multiple times). Define: -
# freq(p) = number of times point p appears in the array.
# - dist(p) = squared Euclidean distance to the origin,
# x^2 + y^2. Return **k distinct points** sorted by the
# following priority: 1. Higher frequency first (freq
# descending) 2. If frequencies tie, smaller distance
# first (dist ascending) 3. If still tied, sort by x
# ascending, then y ascending ### Input - points: list of
# length n, each element [x, y] - k: integer, 1 <= k <= (#
# of distinct points) ### Output - A list of k distinct
# points following the ranking above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1 with points arriving over time.
# 2. Q2 with timestamps arriving out of order.
# 3. Q2 returns the closest value at or before the
# timestamp (LeetCode 981).
#
# INTERVIEW NOTES
# Q1 is a ranking: say the full sort key out loud, count
# with a hash map, and then you only need the K best
# (heap), not a full sort. Q2 keeps each key's timestamps
# in a sorted list, so the nearest one is next to the
# insertion point: binary search, then compare the two
# neighbours.
#
# BRUTE FORCE (say it first): Q1: sort all distinct points
# by the full key
# Count the points, sort the distinct ones by (-frequency,
# distance, x, y), take K. Correct; sorting all d distinct
# points does more work than a heap of size K. Time O(d
# log d). Space O(d).


import heapq
from bisect import bisect_left, bisect_right, insort
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Q1: sort all distinct points
# by the full key | time O(d log d)
def top_points_sort(points, k):
    c = Counter(map(tuple, points))
    return [
        list(p)
        for p in sorted(
            c,
            key=lambda p: (
                -c[p],
                p[0] ** 2 + p[1] ** 2,
                p[0],
                p[1],
            ),
        )[:k]
    ]


# BASE QUESTION (optimal): Q1: heap with the full key
# Idea: Count with a Counter, then heapq.nsmallest(k, ...)
# with the key (-frequency, squared distance, x, y).
# Squared distances avoid square roots and are exact for
# integers.
def top_points(points, k):
    # more frequent first, then closer to the origin, then
    # x, y
    counts = Counter(map(tuple, points))

    def rank(p):
        dist = p[0] ** 2 + p[1] ** 2
        return (-counts[p], dist, p[0], p[1])

    best = heapq.nsmallest(k, counts, key=rank)
    return [list(p) for p in best]


# PART 2: closest timestamp value
class TimeStore:
    """set(key, value, ts) and closest-time reads."""

    def __init__(self):
        # key -> sorted [(timestamp, value)]
        self.data = defaultdict(list)
```

```
    def set(self, key, value, ts):
        # insort also handles timestamps arriving out of
        # order
        insort(self.data[key], (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
        if not items:
            return ""
```

```
        # the first stored time at or after ts, and the
        # one before
        i = bisect_left(items, (ts,))
        options = items[max(0, i - 1):i + 1]
        # closest wins; on a tie the smaller timestamp
        # wins
        def gap(item):
            return (abs(item[0] - ts), item[0])
        best = min(options, key=gap)
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value not after ts
        items = self.data.get(key, [])
        i = bisect_right(items, (ts, chr(0x10FFFF))) - 1
        return items[i][1] if i >= 0 else ""


# FOLLOW-UP 1: Q1 with points arriving over time.
# Answer: Keep a running Counter and compute the top K on
# demand with a heap (O(d log k) per query), or maintain a
# sorted structure updated on every add if queries
# dominate.
class TopPointsStream:
    # follow-up: points arrive one at a time. The counts
    # are
    # kept as points arrive, so top_k never recounts
    # history.
    def __init__(self):
        self.counts = Counter()

    def add(self, x, y):
        self.counts[(x, y)] += 1

    def top_k(self, k):
```

```
        def rank(p):
            dist = p[0] ** 2 + p[1] ** 2
            return (-self.counts[p], dist, p[0], p[1])

        best = heapq.nsmallest(k, self.counts, key=rank)
        return [list(p) for p in best]


if __name__ == "__main__":
    pts = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]
    # (1,1) and (2,0) appear twice; (2,0) is farther? 4 vs
    # 2
    assert top_points(pts, 2) == [[1, 1], [2, 0]]
    four = top_points(pts, 4)
    assert four == [[1, 1], [2, 0], [0, 2], [3, 3]]
    assert top_points([[5, 5]], 1) == [[5, 5]]
    live = TopPointsStream()
    for x, y in pts:
        live.add(x, y)
    assert live.top_k(2) == [[1, 1], [2, 0]]
    live.add(3, 3)
    live.add(3, 3)
    live.add(3, 3)
    # (3, 3) is now the most frequent point
    assert live.top_k(1) == [[3, 3]]
    s = TimeStore()
    for ts, v in ((2, "x"), (10, "y"), (15, "z")):
        s.set("a", v, ts)
    assert s.get_closest("a", 11) == "y"
    assert s.get_closest("a", 14) == "z"
    # a tie: 10 and 15 are both 2.5 away from 12.5, 12 ->
    # 10
    assert s.get_closest("a", 12) == "y"
    assert s.get_closest("b", 1) == ""
    assert s.get_closest("a", 100) == "z"
    assert s.get_closest("a", -5) == "x"
    assert s.get_at_or_before("a", 14) == "y"
```

```
    assert s.get_at_or_before("a", 1) == ""
    s.set("a", "late", 5)
    assert s.get_closest("a", 6) == "late"
```

FUNCTION AND TEST CASES

Signature

```
def solution(points: list[list[int]], k: int) -> list[list
[int]]:
```

Sample call

```
solution([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
```

Test cases

Input	Expected
([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)	[[2, 2], [1, 1]]
([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1] ,[3,0]], 3)	[[0, 2], [1, 2], [2, 1]]

STUDY PATH: question 112 of 167 (more practice)

Priority score 2.28 | Heaps and top-k | asked 225 days ago

MEDIUM | PracHub | Technical Screen | 3 of 4

Determine if a 14-tile hand is winning

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 149/228

ABOUT THIS QUESTION

Topic: More practice - Heaps and top-k

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #659 Split Array into Consecutive Subsequences

LeetCode match: No direct match; related LC 659 Split Array into Consecutive Subsequences

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/determine-if-a-14-tile-hand-is-winning

QUESTION

You are given an integer array **`tiles`** of length 14 representing a Mahjong-like hand. Each integer is a tile value from 1 to 9 (single suit). You may reorder the tiles. A hand is **winning** if it can be partitioned into:

- **1 pair**: two identical tiles, and - **4 melds** (each meld uses 3 tiles), where each meld is either: - a **triplet**: three identical tiles (e.g., 7,7,7), or - a **sequence**: three consecutive values (e.g., 2,3,4). Each tile can be used at most once. Return **`true`** if the hand is winning, otherwise return **`false`**. Assumptions/constraints:
- **`tiles.length == 14`**
- **`1 <= tiles[i] <= 9`**
- Each value can appear multiple times (as in a real tile set).

APPROACH

Count array of tiles 1..9; try each value as the pair, then backtrack: take the smallest remaining tile as a triplet or a run of three. Tiny search space.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether sequences are only consecutive in one suit (values 1 to 9), whether a tile can be used only once, and whether other special hands exist (assume not).

BRUTE FORCE TO SAY OUT LOUD

Try every permutation of the 14 tiles and test the grouping. Far too slow.

OPTIMAL IDEA

Keep a count array of tiles 1 to 9. Try each value that has at least two tiles as the pair, remove it, then backtrack on the remaining 12 tiles: take the smallest value that still has tiles and either remove a triplet of it or a run of three consecutive values starting at it; if neither is possible, fail. Always handling the smallest value makes the search small. Time is tiny for 14 tiles.

DRY-RUN THESE CASES

Four triplets and a pair, four runs and a pair, a hand with two possible groupings, a hand with a pair but impossible melds, all 14 tiles equal (invalid since each value has at most four copies).

LIKELY FOLLOW-UPS

More suits, seven pairs, or all possible winning tiles for a 13-tile hand.

PITFALLS

Greedy without backtracking: a run versus a triplet choice can matter. Forgetting to restore counts.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Work with counts per value (at most nine numbers), not with the tile list. Try every value as the pair, then handle the lowest remaining value: it must belong to a triplet or to a sequence starting at it (nothing smaller is left to pair with it). Always processing the smallest value makes the search tiny and avoids trying the same grouping in different orders.

Basis: Written; a small exhaustive search like the Mahjong 'standard hand' check. Rule: 14 tiles of one suit with values 1..9 split into one pair and four melds, each meld a triplet (x, x, x) or a sequence (x, x + 1, x + 2).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(counts):
    for v in range(1, 10):
        if counts[v] >= 2:
            counts[v] -= 2  # the pair
            ok = remove_melds(counts)
            counts[v] += 2
            if ok:
                return True
    return False
def remove_melds(counts):
    v = first_value_with_tiles(counts)
    if v is None:
        return True
    for meld in (triple(v), run(v)):
        if can_take(counts, meld):
            take(counts, meld)
            ok = remove_melds(counts)
            give_back(counts, meld)
            if ok:
                return True
    return False
```

TIME	SPACE
exponential but small for...	**O(depth)**
exponential but small for 14 tiles	

BRUTE FORCE

WHY IT IS SLOW

It tries the same groupings in many different orders (still tiny for 14 tiles). Always grouping the smallest remaining tile tries each grouping once.

OPTIMAL SOLUTION · Counts, try each pair, then always group the smallest remaining tile

TIME	SPACE
O(9 * small branching) = constant for 14 tiles	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

FOLLOW-UP 2 More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

FOLLOW-UP 3 Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine if a 14-tile hand is winning
#
# You are given an integer array tiles of length 14
# representing a Mahjong-like hand. Each integer is a tile
# value from 1 to 9 (single suit). You may reorder the
# tiles. A hand is **winning** if it can be partitioned
# into: - **1 pair**: two identical tiles, and - **4
# melds** (each meld uses 3 tiles), where each meld is
# either: - a **triplet**: three identical tiles (e.g.,
# 7,7,7), or - a **sequence**: three consecutive values
# (e.g., 2,3,4). Each tile can be used at most once.
# Return true if the hand is winning, otherwise return
# false. Assumptions/constraints: - tiles.length == 14 - 1
# <= tiles[i] <= 9 - Each value can appear multiple times
# (as in a real tile set).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which tile would complete a 13-tile hand?
# 2. More suits or special hands (seven pairs).
# 3. Why is the 'smallest remaining value' rule enough?
#
# INTERVIEW NOTES
# Work with counts per value (at most nine numbers), not
# with the tile list. Try every value as the pair, then
# handle the lowest remaining value: it must belong to a
# triplet or to a sequence starting at it (nothing smaller
# is left to pair with it). Always processing the smallest
# value makes the search tiny and avoids trying the same
# grouping in different orders.
#
# BRUTE FORCE (say it first): Remove any triplet or
# sequence from anywhere, exhaustively
# For each pair candidate, repeatedly try removing any
# triplet or any sequence available among the counts (in
# any order) and see whether everything can be removed.
# Correct and the oracle; it tries the same groupings in
# many orders, unlike the smallest-first search. Time
# exponential but small for 14 tiles. Space O(depth).


from collections import Counter


# BRUTE FORCE (say it first): Remove any triplet or
# sequence from anywhere, exhaustively | time exponential
# but small for 14 tiles
def winning_brute(tiles):
    cnt = [0] * 12
    for t in tiles:
        cnt[t] += 1

    def melds(c):
        if sum(c) == 0:
            return True
        for v in range(1, 10):
            if c[v] >= 3:
                c[v] -= 3
                ok = melds(c)
                c[v] += 3
                if ok:
                    return True
            if (
                v <= 7
                and c[v]
                and c[v + 1]
                and c[v + 2]
            ):
                c[v] -= 1
                c[v + 1] -= 1
                c[v + 2] -= 1
                ok = melds(c)
                c[v] += 1
                c[v + 1] += 1
                c[v + 2] += 1
                if ok:
                    return True
        return False

    for v in range(1, 10):
        if cnt[v] >= 2:
            cnt[v] -= 2
            ok = melds(cnt)
            cnt[v] += 2
            if ok:
                return True
    return False


# HELPERS for the base question
def melds_ok(cnt, v=1):
```

```
    # can the tiles in cnt (value -> count) form only
    # melds?
    while v <= 9 and cnt[v] == 0:
        v += 1
    if v > 9:
        return True
    # the smallest tile is either in a triplet ...
    if cnt[v] >= 3:
        cnt[v] -= 3
        ok = melds_ok(cnt, v)
        cnt[v] += 3
        if ok:
            return True
    # ... or the start of a run v, v + 1, v + 2
    if v <= 7 and cnt[v + 1] and cnt[v + 2]:
        for t in (v, v + 1, v + 2):
            cnt[t] -= 1
        ok = melds_ok(cnt, v)
        for t in (v, v + 1, v + 2):
            cnt[t] += 1
        return ok
    return False


# HELPERS for follow-up 2
def count_of(values):
    cnt = Counter({v: 0 for v in range(1, 12)})
    for v in values:
        cnt[v] += 1
    return cnt


def is_winning_hand_with_len(values):
    cnt = count_of(values)
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False


# BASE QUESTION (optimal): Counts, try each pair, then
```

```
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair.
def is_winning_hand(tiles):
    # one pair plus four melds, single suit, values 1..9
    cnt = Counter({v: 0 for v in range(1, 12)})
    for t in tiles:
        cnt[t] += 1
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False


# FOLLOW-UP 1: Which tile would complete a 13-tile hand?
# Answer: Try each of the nine values as the 14th tile and
# run the same check: 9 checks of a tiny search. That is
# how a player finds the tiles they are waiting for.
def waiting_tiles(tiles13):
    # follow-up: which 14th tile completes the hand?
    return [t for t in range(1, 10)
            if is_winning_hand(tiles13 + [t])]
```

```
# FOLLOW-UP 2: More suits or special hands (seven pairs).
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
def is_seven_pairs(tiles):
    # a special hand: seven different pairs
    counts = Counter(tiles)
    pairs = all(c == 2 for c in counts.values())
    return len(tiles) == 14 and pairs


# FOLLOW-UP 2: More suits or special hands (seven pairs).
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
```

```
def is_winning_multi(hand):
    # several suits: hand is a list of (suit, value);
    # melds
    # never cross suits, and exactly one suit holds the
    # pair
    suits = {}
    for suit, v in hand:
        suits.setdefault(suit, []).append(v)
    pair_suits = [s for s in suits
                  if len(suits[s]) % 3 == 2]
    if len(pair_suits) != 1:
        return False
    for s, vs in suits.items():
        if s == pair_suits[0]:
            if not is_winning_hand_with_len(vs):
                return False
        elif len(vs) % 3 or not melds_ok(count_of(vs)):
            return False
    return True


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
    assert is_winning_hand(win)
    # 3 and 6 are missing, so the runs cannot be formed
    lose = [1, 2, 4, 5, 7, 8, 1, 2, 4, 5, 7, 8, 9, 9]
    assert not is_winning_hand(lose)
    # thirteen tiles waiting for a 9 (or a 6)
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]
    assert 9 in waiting_tiles(wait)
    assert is_seven_pairs([1, 1, 2, 2, 3, 3, 4, 4, 5, 5,
                           6, 6, 7, 7])
    hand = [("m", v) for v in (1, 1, 1, 2, 3)] + [
        ("p", v) for v in (4, 5, 6, 7, 7, 7)] + [
        ("s", v) for v in (2, 2, 2)]
    assert is_winning_multi(hand)
```

FUNCTION AND TEST CASES

Signature

```
def solution(tiles: list[int]) -> bool:
```

Sample call

```
solution([1,1,1,2,3,4,2,3,4,5,6,7,9,9])
```

Test cases

Input	Expected
([1,1,1,2,3,4,2,3,4,5,6,7,9,9],)	True
([2,2,2,3,3,3,4,4,4,5,5,5,9,9],)	True

ALSO REPORTED · 1 more

Mahjong Winning Hand Detection (14 Tiles) | 1Point3Acres | onsite-coding | asked 2026-07-20 | Low

https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:
Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.
For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

Preparation: Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY PATH: question 113 of 167 (more practice)

Priority score 0.7 | Heaps and top-k | asked 286 days ago

HARD | PracHub | Technical Screen | 4 of 4

Return top-k subset sums in descending order

POSTED 2026-01-07 | 286 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 170/228

ABOUT THIS QUESTION

Topic: More practice - Heaps and top-k

Optimal answer from: the interview site's own solution

LeetCode match: LC 2386 Find the K-Sum of an Array

Posted on PracHub: 2026-01-07 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/return-top-k-subset-sums-in-descending-order

QUESTION

Overview: This question evaluates proficiency in combinatorial reasoning, algorithmic optimization for large search spaces, and efficient extraction of top-k values within the coding & algorithms domain.

APPROACH

Sort values, start from the all-positives sum, and use a max/min-heap of (sum, index) states that either drop the next element or swap it for the following one, popping k times. O(n log n + k log k).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the empty subset counts, whether values can be negative or repeated, how large k is compared with 2^n, and whether equal sums count separately.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all 2^n subsets, sort, take k. Fine for n <= 20; say that out loud.

OPTIMAL IDEA

Reduce negatives: let base be the sum of the positive values, and work with absolute values. Every subset sum of the original equals base minus a subset sum of the absolute values, so the k largest sums are base minus the k smallest subset sums of the sorted absolute values. Generate the k smallest sums with a min-heap of (sum, last index): from a state you either add the next element or replace the last element with the next one. Each pop yields the next smallest sum and creates at most two states, so O(n log n + k log k).

DRY-RUN THESE CASES

All positive values. All negative values (best is the empty subset). A zero in the array. k = 1 and k = 2^n.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Need k sums without storing all of them, subsets themselves (store parent pointers), or a limit on subset size.

PITFALLS

Forgetting to sort first, which breaks the add-next/replace-last move set. Pushing duplicate states.

HOW TO THINK ABOUT IT

k is small compared with 2^n, so never enumerate all subsets. Turn the problem into 'k smallest sums of non-negative numbers': subtract everything from the sum of the positive values. Then grow answers in increasing order with a heap where each state has exactly two successors (extend with the next element, or swap the last element for the next one).

Basis: Written; the classic 'k smallest subset sums' heap technique (also behind LeetCode 2386, K-Sum of an Array).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    sums = []
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            sums.append(sum(subset))
    sums.sort(reverse=True)
    return sums[:k]
```

TIME	SPACE
O(2^n * n)	O(2^n)

WHY IT IS SLOW

2^n subsets and a sort of them: O(2^n * n) time and O(2^n) space, hopeless past about n = 20. A heap that grows the k best sums step by step is O(n log n + k log k).

OPTIMAL SOLUTION · Heap of the k smallest sums of absolute values

TIME	SPACE
O(n log n + k log k)	O(n + k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

FOLLOW-UP 2 **k can be as large as 2^n.**

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

FOLLOW-UP 3 **Limit subsets to at most m elements.**

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return top-k subset sums in descending
# order
#
# This question evaluates proficiency in combinatorial
# reasoning, algorithmic optimization for large search
# spaces, and efficient extraction of top-k values within
# the coding & algorithms domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsets themselves, not only the sums.
# 2. k can be as large as 2^n.
# 3. Limit subsets to at most m elements.
#
# INTERVIEW NOTES
# k is small compared with 2^n, so never enumerate all
# subsets. Turn the problem into 'k smallest sums of non-
# negative numbers': subtract everything from the sum of
# the positive values. Then grow answers in increasing
# order with a heap where each state has exactly two
# successors (extend with the next element, or swap the
# last element for the next one).
#
# BRUTE FORCE (say it first): Enumerate all 2^n subsets
# and sort
# Compute every subset sum, sort in descending order and
# take k. Fine for n up to about 20, hopeless beyond that.
# Time O(2^n * n). Space O(2^n).


import heapq
from itertools import combinations


def brute(nums, k):
    # every subset: the check used by the demo
    sums = []
    for r in range(len(nums) + 1):
        for c in combinations(nums, r):
            sums.append(sum(c))
    return sorted(sums, reverse=True)[:k]


# BRUTE FORCE (say it first): Enumerate all 2^n subsets
# and sort | time O(2^n * n)
def top_k_sums_brute(nums, k):
```

```
    sums = []
    for mask in range(1 << len(nums)):
        sums.append(
            sum(
                nums[i]
                for i in range(len(nums))
                if mask >> i & 1
            )
        )
    return sorted(sums, reverse=True)[:k]


# BASE QUESTION (optimal): Heap of the k smallest sums of
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values.
def top_k_sums(nums, k):
    # Idea: start from the best subset (all positives).
    # Every
    # other subset is that one with some elements
    # "flipped",
    # which costs |x|. So the k best subsets are the k
    # cheapest sets of flips: a heap over sorted |x|.
    base = sum(x for x in nums if x > 0)
    cost = sorted(abs(x) for x in nums)
    out = [0]
    heap = [(cost[0], 0)] if cost else []
    while heap and len(out) < k:
        s, i = heapq.heappop(heap)
        out.append(s)
        if i + 1 < len(cost):
            # two successors, neither is ever cheaper:
            # add the next element ...
            heapq.heappush(heap, (s + cost[i + 1], i + 1))
            # ... or move the last flipped one up by one
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1))
    return [base - c for c in out[:k]]


# FOLLOW-UP 1: Return the subsets themselves, not only the
# sums.
# Answer: Store each heap state as a node with (last
```

```
# index, parent node) instead of copying lists, and
# rebuild a subset by walking parents only for the k
# answers. That keeps pushes O(1).
def top_k_subsets(nums, k):
    # also return the subsets: states keep a link to their
    # parent instead of copying lists
    order = sorted(range(len(nums)),
                   key=lambda i: abs(nums[i]))
    cost = [abs(nums[i]) for i in order]
    best = {i for i in range(len(nums)) if nums[i] > 0}

    def subset(chain):
        flipped = set()
        while chain is not None:
            flipped.add(order[chain[0]])
            chain = chain[1]
        return sorted(nums[i] for i in best ^ flipped)

    base = sum(nums[i] for i in best)
    out = [(base, sorted(nums[i] for i in best))]
    heap = [(cost[0], 0, (0, None))] if cost else []
    while heap and len(out) < k:
        s, i, chain = heapq.heappop(heap)
        out.append((base - s, subset(chain)))
        if i + 1 < len(cost):
            heapq.heappush(heap, (s + cost[i + 1], i + 1,
                                  (i + 1, chain)))
            # replace the last flip i by i + 1
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1,
```

```
                                (i + 1, chain[1])))
    return out[:k]


if __name__ == "__main__":
    nums = [3, -1, 2, -4]
    assert top_k_sums(nums, 5) == brute(nums, 5)
    assert top_k_sums(nums, 4) == [5, 4, 3, 2]
    # k bigger than the number of subsets: all 2^n sums
    assert top_k_sums([1, 2], 10) == brute([1, 2], 10)
    assert top_k_sums([], 3) == [0]
    cases = ([5, -2, 7, 1], [-1, -2, -3], [0, 4, -4])
    for sample in cases:
        for k in range(1, 2 ** len(sample) + 1):
            got = top_k_sums(sample, k)
```

```
            assert got == brute(sample, k)
    # the subsets add up to the sums
    for total, sub in top_k_subsets(nums, 4):
        assert sum(sub) == total
    totals = [t for t, _ in top_k_subsets(nums, 4)]
    assert totals == [5, 4, 3, 2]
```

FUNCTION AND TEST CASES

Signature

```
def solution(w: list[int], k: int) -> list[int]:
```

Sample call

```
solution([3, 2, 1], 4)
```

Test cases

Input	Expected
([3, 2, 1], 4)	[6, 5, 4, 3]
([5], 2)	[5, 0]